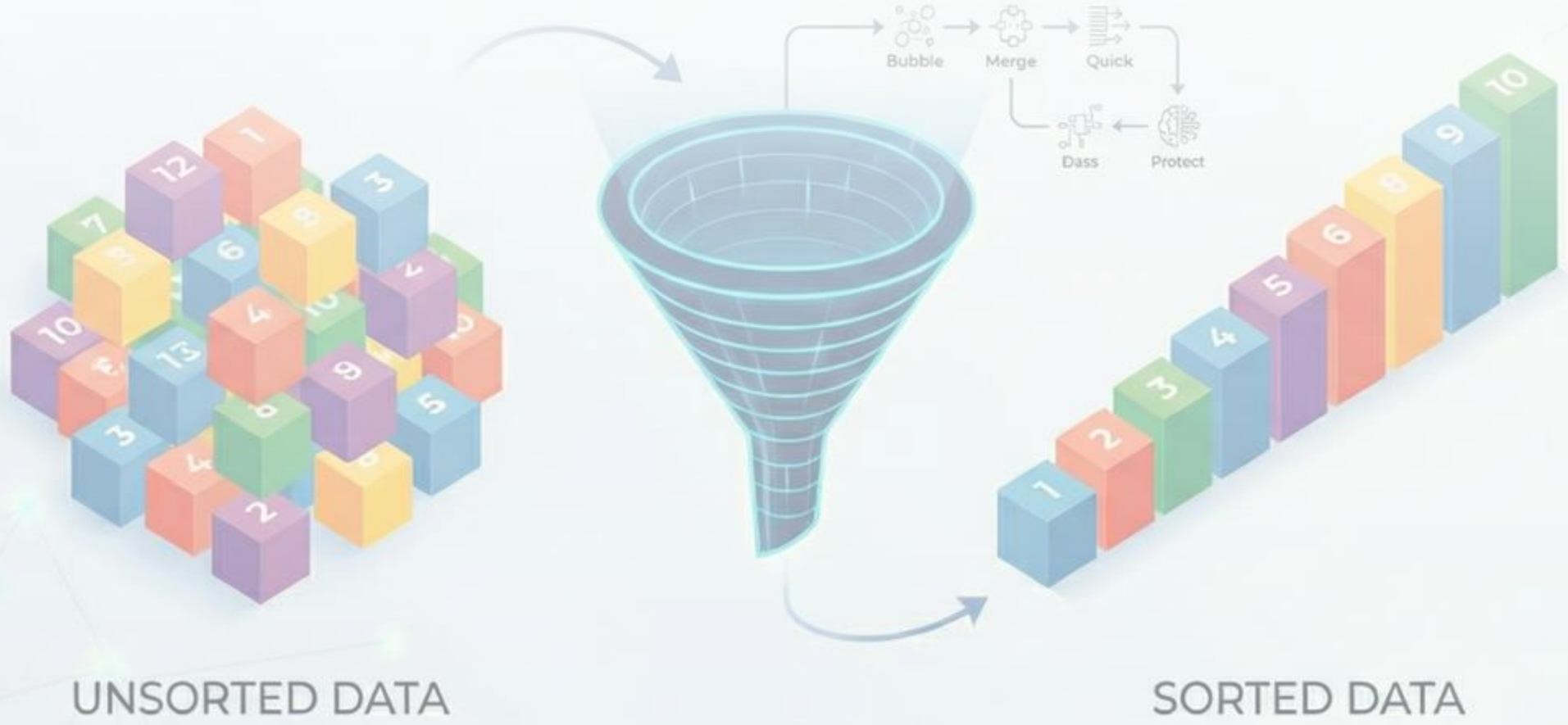
Sorting Algorithms



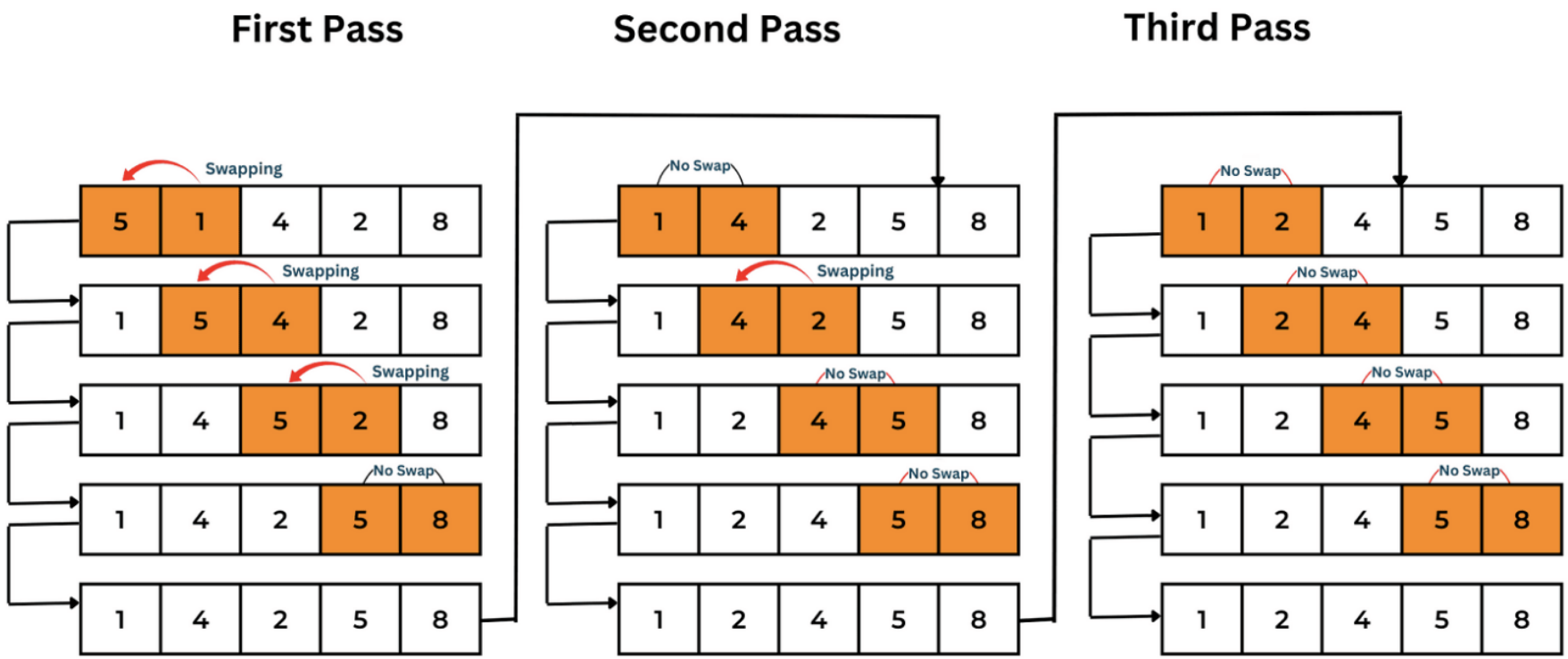
Why sort?

- Organize data: Useful for inspecting data or creating leaderboards (e.g, see which team is ranked first, second, third in the NBA)

- Sorting is a pre-processing step for other algorithms (binary search, Kruskal's algorithm)

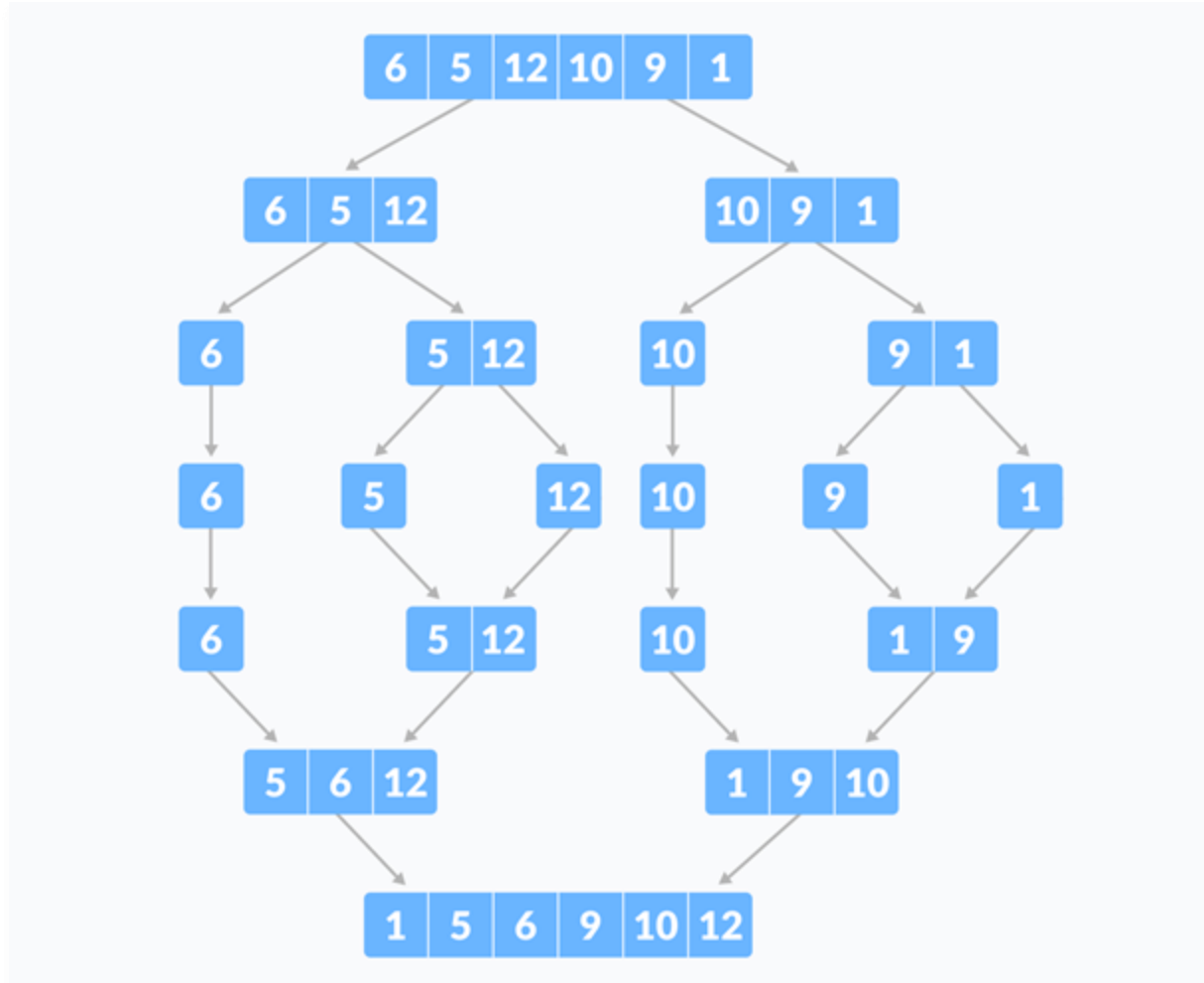
Bubblesort

- Repeatedly steps through the list
- Compares adjacent elements
- Swaps them if they are in the wrong order
- Largest elements "bubble" to the top.


First Pass
Second Pass
Third Pass
Swapping
No Swap
5 1 4 2 8
1 5 4 2 8
1 4 5 2 8
1 4 2 5 8
1 4 2 5 8
1 4 2 5 8
1 2 4 5 8
1 2 4 5 8
1 2 4 5 8
1 2 4 5 8
1 2 4 5 8
1 2 4 5 8
1 2 4 5 8

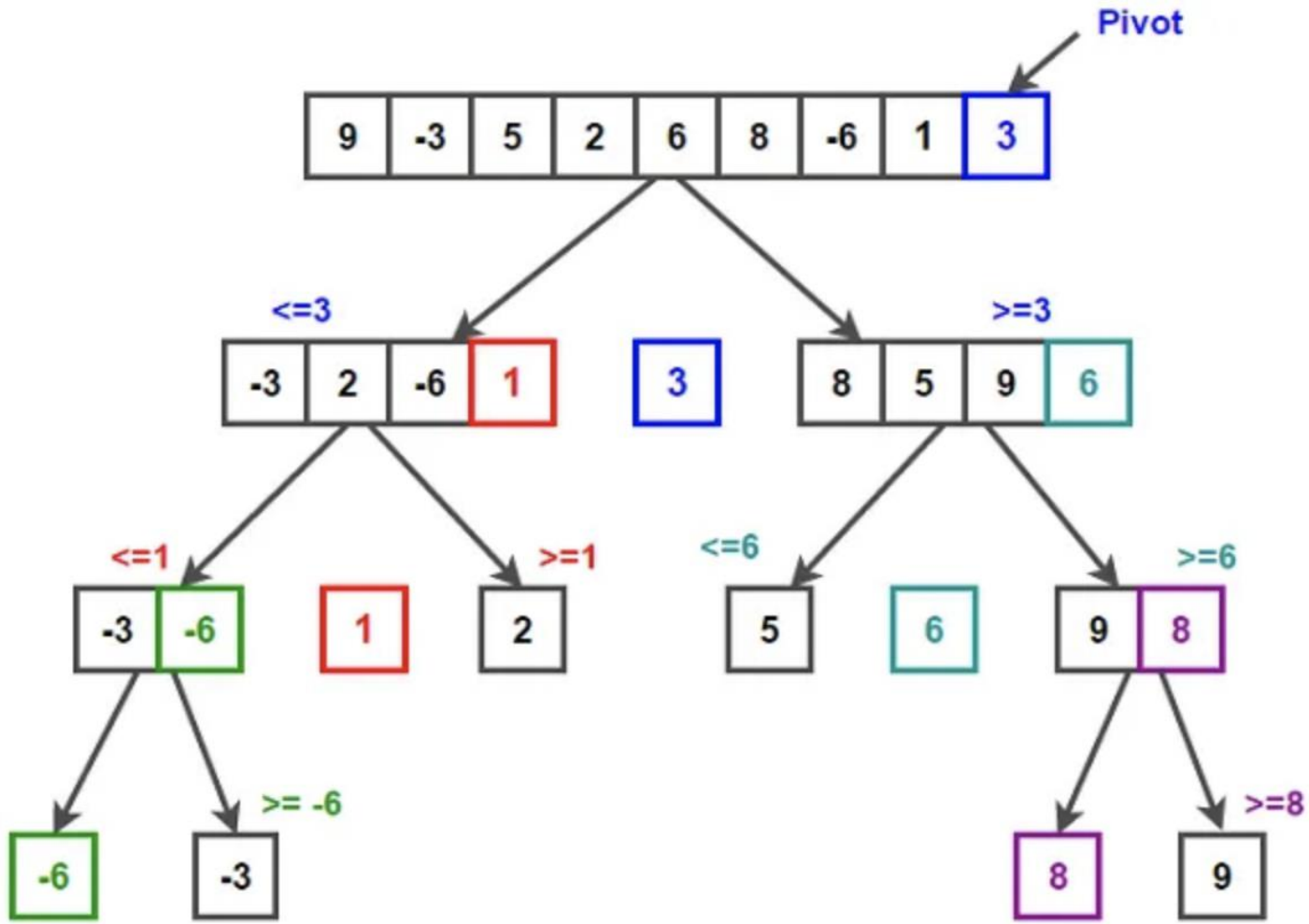
Mergesort

- "Divide and Conquer" algorithm
- Split (or… divide) the list in half
- Sort the two halves recursively
- Merge the two halves (…and you have conquered the problem)


6 5 12 10 9 1
6 5 12
10 9 1
6
5 12
10
9 1
6
5
12
10
9
1
6
5 12
10
1 9
5 6 12
1 9 10
1 5 6 9 10 12

Quicksort

- **Pivot:** Pick a 'pivot' point.
- **Partition:** Move smaller items to the left of the pivot, and larger items to the right.
- **Repeat:** Apply recursively.
- Depends highly on pivot choice, but with decent probability, we can choose a good pivot


Pivot
9 -3 5 2 6 8 -6 1 3
<=3
-3 2 -6 1
3
>=3
8 5 9 6
<=1
-3 -6
1
2
>=1
<=6
5
6
9 8
>=6
-6
-3
>= -6
8
9
>=8

Which algorithm to choose?

- In practice, quicksort (as the name suggests…) is the fastest and works for any type of data

- Some **non-comparison-based algorithms** can be used to sort specific data types, such as integers. See Counting Sort, Radix Sort, Bucket Sort. These are faster but only work for certain data types.

- **Question:** If you had 50 teddy bears in front of you, how would you, as a human, sort them in increasing size? How fast? Does your answer change if you are given 100, 1K, or 1M teddy bears?